

# Generación de imágenes artificiales de leucocitos usando una vanilla GAN.

Proyecto final

Rosa Atenea Hedone Rangel Quiroz. NUA: 603171. Correo: rah.rangelquiroz@ugto.mx

## **Introducción.**

El conteo manual de glóbulos blancos en la sangre se hace por medio un hemo citómetro (cámara de Neubauer) y un microscopio. Evaluar de manera directa la cantidad y morfología de los glóbulos blancos en la sangre permite hacer un diagnóstico primario o para verificar resultados automatizados anormales, infecciones, inflamaciones o anomalías celulares.

Está técnica incluso siendo realizada por profesionales entrenados es una técnica que consume mucho tiempo y tiende mucho a fallas preanalíticas (toma de muestra y dilución) y errores de procedimiento. También es importante tomar en cuenta que una mala técnica suele alterar drásticamente el resultado.

Como una alternativa al conteo de leucocitos manual se ha propuesto entrenar redes neuronales de aprendizaje profundo que puedan realizarlo, es ahí donde la falta de datos con las que entrenar estás redes entra a la problemática. La propuesta de este proyecto es implementar una Vanilla Gan, es decir, la primera estructura de redes generativas antagónicas, para generar imágenes de leucocitos para uso de aprendizaje médico

## **Objetivo.**

Implementar, desarrollar y entrenar una red generativa antagónica tomando como referencia la arquitectura vanilla GAN y la teoría usada en (Sangal, Krishnan, 2022) para reproducir imágenes generales de glóbulos blancos.

## **Justificación.**

La detección de enfermedades e infecciones en la sangre implica un problema de visualización y obtención de datos para analizar en la actualidad. Es por esto, que previo a los problemas que implican el trato directo con los datos médicos se propone empezar por generar más datos. Es así como las redes generativas aportan una solución viable a la generación de datos que se requieren. Es

importante que además de aumentar la cantidad de datos aportan mejor visualización a la morfología que compone cada célula de la sangre.

Es importante justificar también que este problema carece de información y réplicas, por lo que el entrenamiento y personalización de la red neuronal generativa es clave para observar mejores resultados, dicha mejora es gradual y la perfección de las imágenes se puede observar comparando los resultados de diferentes arquitecturas generativas. En este proyecto se ha implementado una de estas arquitecturas tomando como base la información reportada en Sangal, R., & Sankaran Krishnan, V. ([2022]). *Enhancing white blood cell classification through GAN-based data augmentation* [Reporte de proyecto no publicado].

### Marco teórico.

Un leucocito o glóbulo blanco es una célula nucleada del sistema inmunitario que carece de hemoglobina, siendo esta célula una de las tres que componen la sangre y se produce específicamente en la médula ósea.

Estas células se clasifican según la presencia de gránulos en su citoplasma y también por la forma de su núcleo, siendo dos posibles clasificaciones:

- **Granulocitos (polimorfonucleares)**

Núcleos irregulares divididos en lóbulos y gránulos visibles en su citoplasma. Se componen de **Neutrófilos**, **Eosinófilos**, **Basófilos**.

- **Agranulocitos (mononucleares)**

Núcleo simple, redondeado o con muescas, carecen de gránulos específicos evidentes. Se componen principalmente de **Linfocitos**, **Monocitos**.



FIGURA M.1: CLASIFICACIÓN DE LEUCOCITOS PARA VISUALIZAR LA MORFOLOGÍA.

Se menciona la clasificación de los leucocitos dado que como se puede observar cada uno presenta una morfología distinta y según el tipo que aparezca en las imágenes con las que se entrena la red neuronal, será el tipo más parecido de morfología que se obtendrá también en los resultados sintéticos. El análisis dirigido hacia la morfología del leucocito es una tarea que requiere la visión de especialistas y análisis más profundo. En este proyecto se busca plantear la obtención de una imagen sintética de un leucocito general con el propósito de observar que tan cercana es a una real.

El pronombre Vainilla hace referencia a estándar o base, esto porque la vainilla Gan es la primera arquitectura de red generativa antagónica (GAN por sus siglas en inglés) que funciona como estructura principal para las redes generativas modificadas que fueron usadas después con motivos de optimización. Incluso este proyecto uso toda la arquitectura de una vainilla GAN con personalizaciones en algunos parámetros para optimizar los resultados. La estructura de una Gan fue descrita por primera vez por Ian Goodfellow cómo:

*" Un nuevo marco para estimar modelos generativos mediante un proceso adversario, en el que entrenamos simultáneamente dos modelos: un modelo generativo  $G$  que captura la distribución de los datos y un modelo discriminativo  $D$  que estima la probabilidad de que una muestra provenga de los datos de entrenamiento en lugar de  $G$ . El procedimiento de entrenamiento para  $G$  consiste en maximizar la probabilidad de que  $D$  cometa un error." (Goodfellow, 2014)*

Donde al profundizar podemos observar que se establece el Generador, denotado como  $G$ . El objetivo de  $G$  no es memorizar un conjunto de datos, sino aprender su distribución de probabilidad subyacente, definida como  $p_g$ . El proceso de generación comienza con la introducción de variables de ruido aleatorio, representadas como  $z$ , las cuales se extraen de una distribución de probabilidad a priori conocida como  $p_z(z)$ . Este ruido actúa como la materia prima del sistema. El Generador se define matemáticamente como una función diferenciable  $G(z; \theta_g)$ , donde  $\theta_g$  representa los parámetros, o pesos, de su red neuronal. La función de esta red es mapear el vector de ruido  $z$  y proyectarlo hacia el espacio de los datos de alta dimensionalidad, produciendo así una muestra artificial.

Una vez que se ha definido el generador se establece el Discriminador, denotado como  $D$ , que funciona como un clasificador binario. Esta segunda red neuronal posee sus propios parámetros, definidos como  $\theta_d$ . La función del Discriminador, expresada como  $D(x; \theta_d)$ , toma como entrada un dato  $x$ . Este dato  $x$  puede

provenir directamente del conjunto de entrenamiento legítimo o puede ser una de las muestras artificiales producidas por el Generador. La red procesa esta entrada y devuelve un único valor escalar. Este valor escalar,  $D(x)$ , representa la probabilidad calculada por la red de que el dato analizado provenga de la distribución de datos reales en lugar de la distribución artificial  $p_g$  creada por el Generador.

La estructura modelo reside en el entrenamiento simultáneo de ambas redes, el cual se estructura con objetivos matemáticos diametralmente opuestos. Por un lado, el Discriminador se entrena para maximizar su precisión. Su meta es asignar de forma correcta las etiquetas correspondientes tanto a las muestras de entrenamiento reales como a las muestras falsas provenientes de  $G$ . Esto significa que la red ajusta sus parámetros  $\theta_d$  para que la salida de  $D(x)$  sea alta para los datos reales y baja para los datos generados.

Dado que la única forma de evaluar la calidad de las muestras del Generador es a través de la evaluación del Discriminador, denotada como  $D(G(z))$  el Generador ajusta sus parámetros  $\theta_g$  para minimizar la expresión matemática  $\log(1 - D(G(z)))$ . En términos prácticos, el Generador busca reducir a cero la confianza del Discriminador al identificar las muestras falsas. Mediante este juego de optimización continua y el cálculo constante de gradientes, ambas redes se impulsan a mejorar progresivamente: el Discriminador se vuelve un evaluador más estricto y el Generador perfecciona su capacidad de crear muestras estadísticamente indistinguibles de los datos originales.

## Diagrama de bloques.

### *I. Dataset.*

#### *• Descripción.*

Para el entrenamiento de esta red neuronal se utilizó el dataset BCCD el cual es de dominio público y se encuentra disponible en la plataforma kaggle desde el año 2022, este fue subido por el propietario Konstantin Azov. Consiste en 364 imágenes de células pertenecientes a la sangre, su nombre es por las siglas “Blood cells count and detection”. El dataset cuenta con cuatro subtipos de leucocitos y en este proyecto decidió usarse el dataset completo sin emplear las clases en las que se encuentran separados cada tipo de leucocitos. La composición del dataset es la siguiente:

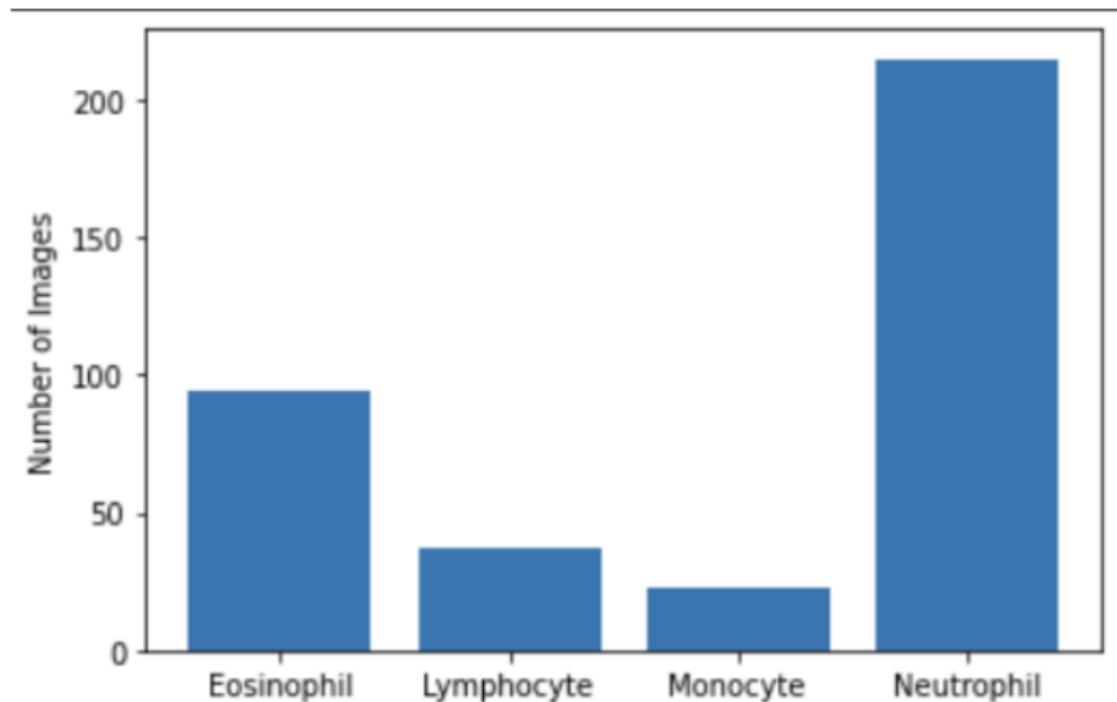


FIGURA D1: COMPOSICIÓN DEL DATASET EMPLEADO.

| Tipo de célula. | Cantidad de imágenes |
|-----------------|----------------------|
| Neutrófilos.    | 215                  |
| Eosinófilos.    | 94                   |
| Linfocitos.     | 37                   |
| Monocitos.      | 23                   |

La cantidad de neutrófilos y monocitos tiene una diferencia de casi 10 veces mayor una que la otra, por lo tanto, es notable que la red aprenderá a generar imágenes de células del tipo granulocitos con una precisión alta.

Cómo se mencionó antes, se empleó el número completo de imágenes puesto en una clase completa correspondiente al nombre “WBC”, white blood cells. Así cómo primera tarea se podrá observar qué tan bien reproduce imágenes generales de células correspondientes a la morfología de un granulocito.

Algunos ejemplos de las imágenes con las que se entrenó la red son:



FIGURA DII: IMAGEN DE ENTRENAMIENTO REAL.



FIGURA DIII: IMAGEN DE ENTRENAMIENTO REAL.

- ***Preparación del dataset.***

Antes de construir la arquitectura del dataset y pasar al preprocesamiento de este, fue necesario organizarlo; esto se hizo usando un csv de metadatos el cual se define como un archivo de texto plano que almacena la información de manera organizada separando cada valor. Este archivo copió cada imagen a una misma carpeta.

- ***Preprocesamiento del dataset.***

Cada imagen del dataset contaba con una resolución de 640 x 480 x 3 pixeles, así que para reducir el costo computacional en la RAM de Colab que fue el ambiente donde se ejecutó la red y también tomando en cuenta que una mayor resolución de las imágenes, al menos en la arquitectura empleada no implica una optimización para el resultado; fue que se redimensionó hacia 64 x 64 x 3 pixeles cada imagen. Después se convierten a RGB para tener consistencia en los canales de color y finalmente se obtiene un tensor de 4D que tiene como primera dimensión el número de imágenes. Con esto normalizamos los pixeles a valores entre  $[-1,1]$  para poder usar la función  $\tanh$  de activación en la arquitectura del generador.

## ***II. Arquitectura de cada parte de la red neuronal.***

A pesar de que este proyecto se basó en la arquitectura de una vanilla Gan, se realizaron personalizaciones con la esperanza de obtener mejores resultados. Cada una de estas personalizaciones se explicarán en cada parte de la arquitectura.

- ***Generador:***

La arquitectura generador comienza con la extracción de un vector de ruido aleatorio de 100 dimensiones desde un espacio latente, el cual actúa como la semilla de información inicial. Este vector unidimensional se proyecta mediante una capa densa y se remodela en un tensor tridimensional para habilitar el procesamiento espacial. A partir de este punto, la arquitectura emplea una expansión progresiva a través de bloques secuenciales que combinan operaciones de *UpSampling* para duplicar iterativamente la resolución y capas convolucionales encargadas de aprender patrones visuales jerárquicos, definiendo desde la estructura geométrica global hasta los detalles morfológicos más finos.

Con el propósito de preservar la fidelidad de las texturas naturales y mitigar la aparición de artefactos visuales, se decidió que el modelo prescindiera de capas de estabilización intermedias como el *BatchNormalization*, delegando el control del gradiente a la optimización mediante entropía cruzada. Finalmente, se llega a una capa convolucional final que colapsa el tensor a tres canales de color (RGB) con una resolución de  $64 \times 64$  píxeles, aplicando una función de activación Tanh para restringir los valores de salida al rango continuo de  $[-1, 1]$ , para asegurar que los datos estén centrados en 0 y evitar que el gradiente genere un sesgo hacia una sola dirección.

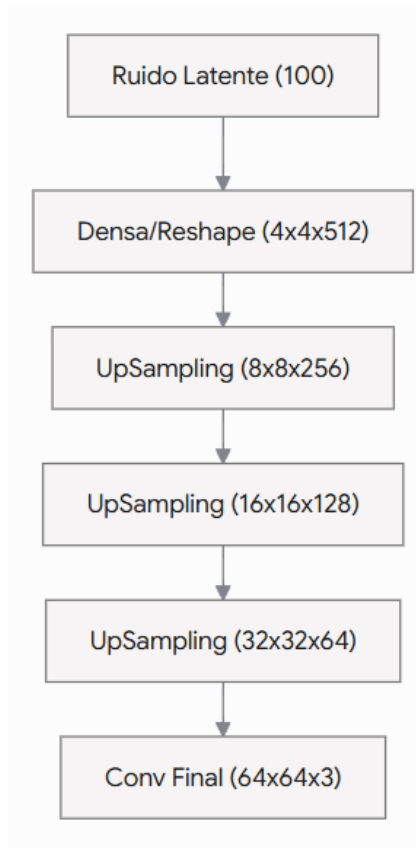


FIGURA AII. ARQUITECTURA DEL GENERADOR.

- ***Discriminador:***

El discriminador opera como un clasificador binario diseñado para evaluar la autenticidad de las imágenes procesadas, diferenciando entre muestras reales y síntesis artificiales. Para llevar a cabo la extracción de características, la arquitectura prescinde de las operaciones destructivas de *pooling* en favor de capas convolucionales con *stride*, esto permite a la red aprender iterativamente su propia estrategia de reducción de dimensionalidad espacial mientras preserva la integridad de la topología y las texturas finas de la imagen. A nivel de activación, se implementa *LeakyReLU* en las capas intermedias; esta función mitiga el fenómeno de los gradientes muertos al permitir una leve propagación de valores negativos, garantizando así un flujo continuo de derivadas hacia el generador durante la retropropagación. Finalmente, con la intención de mantener el equilibrio en la dinámica de optimización adversarial y prevenir que el discriminador converja antes de lo deseado, haciendo que el aprendizaje del generador se anule, se incorporan capas de *Dropout* que actúan como regularizadores, deshabilitando estocásticamente una fracción de las neuronas para forzar el aprendizaje de representaciones características distribuidas y evitar el sobreajuste a los datos de entrenamiento.

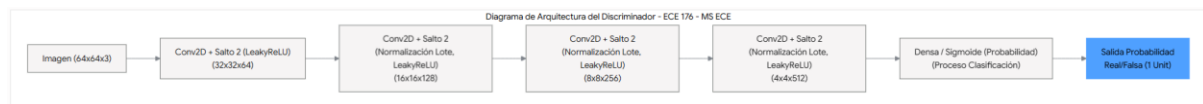


FIGURA AIII. ARQUITECTURA DEL DISCRIMINADOR.

- ***Funciones de pérdida.***

El proceso de optimización de ambas arquitecturas se rige a partir de la función de pérdida de Entropía Cruzada Binaria (*Binary Cross-Entropy* o BCE), esta es una métrica estadística derivada de la teoría de la información que evalúa el rendimiento en tareas de clasificación donde se cuantifica la divergencia logarítmica entre las probabilidades predichas por la red y las etiquetas de clase reales. Dentro del marco de entrenamiento adversarial, la aplicación de la BCE se divide para atender los objetivos matemáticamente contrapuestos de ambos modelos. Mientras que, el discriminador calcula sus gradientes minimizando una pérdida agregada que comprende dos componentes de error específicos: la penalización por clasificar erróneamente una imagen auténtica del conjunto de datos como falsa (evaluando la predicción contra una etiqueta objetivo de 1), y la divergencia por validar incorrectamente una muestra sintética como genuina (evaluando contra una etiqueta objetivo de 0). El generador carece de acceso directo a los datos legítimos e invierte la dinámica de la función; su ciclo de optimización calcula el error emparejando las predicciones que el discriminador emite sobre las imágenes artificiales con una etiqueta objetivo deliberada de 1. En consecuencia, el generador ajusta iterativamente sus pesos mediante retropropagación con el único propósito de minimizar esta pérdida fabricada, forzando a su red a perfeccionar la fidelidad morfológica y textural de sus salidas hasta que logre vulnerar el umbral de decisión del discriminador y engañarlo con éxito.

| Red                     | Objetivo                                     | Etiqueta esperada |
|-------------------------|----------------------------------------------|-------------------|
| Discriminador (reales). | Predecir 1.                                  | 1                 |
| Discriminador (falsas). | Predecir 0.                                  | 0                 |
| Generador.              | Que se prediga 1 en las imágenes sintéticas. | 1                 |

- ***Ciclo de entrenamiento***

Durante cada iteración del proceso de entrenamiento, la arquitectura ejecutó un ciclo en la fase de avance (*forward pass*), el generador sintetiza inicialmente un lote de imágenes artificiales a partir de vectores de ruido latente, tras lo cual el discriminador procesa y evalúa simultáneamente tanto estas muestras sintéticas como un lote equivalente de imágenes reales extraídas del conjunto de datos.

Para gestionar el aprendizaje, la implementación computacional hace uso de la API **tf.GradientTape** de TensorFlow, una herramienta de diferenciación automática que registra las operaciones matemáticas en la memoria (Grafo computacional). Esta separación es la que permite calcular y aplicar la retropropagación de los gradientes de manera estrictamente independiente para los parámetros  $\theta_g$  y  $\theta_d$  : esto asegura que la actualización de los pesos la red no contamine el espacio de optimización de la otra. Finalmente, con intención de maximizar la eficiencia, toda la lógica de este paso de entrenamiento se encapsula bajo el decorador **@tf.function**, esto funciona como una directiva que instruye al *framework* a compilar el código dinámico de Python y transformarlo en un grafo estático de operaciones de bajo nivel, eliminando el sobre costo de ejecución interpretada, también facilita la paralelización en la GPU y acelera de forma drástica los tiempos de convergencia del modelo.

Finalmente, los parámetros con lo que se trabajó de manera final fueron:

| Parámetro. | Valor. | Motivación.                                                                                                                                                                          |
|------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Batch_size | 128    | El batch size fue un parámetro que se modificó durante varias ocasiones en diferentes ciclos de entrenamiento con el propósito de observar el avance en las imágenes sintetizadas.   |
| Latent_dim | 100    | Este es un espacio latente estándar y su modificación no se encontró de mejora en el entrenamiento, así que se quedó estable cada ciclo.                                             |
| Epochs     | 2000   | El número de épocas sí fue otro parámetro que varió durante diferentes ciclos de entrenamiento con la intención de nuevo, de visualizar cada mejora durante el aprendizaje de la red |

|               |        |                                                                                                                                                                                                                    |
|---------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               |        | neuronal. Siento 2000 el número óptimo antes de que el discriminador aprendiera de memoria cada píxel de las imágenes reales truncando el aprendizaje del generador y resultará en imágenes lejanas a lo esperado. |
| Learning_rate | 0.0002 | Valor estándar que se propone para asegurar estabilidad en el aprendizaje.                                                                                                                                         |
| Beta          | 0.5    | Valor estándar para controlar la tasa de decaimiento y estabilizar el entrenamiento de la red.                                                                                                                     |

### ***III. Resultados.***

Para observar el avance en el aprendizaje de la red se modificaron parámetros durante 6 ciclos de entrenamiento, así fue posible monitorear el aprendizaje gradual en la generación de imágenes sintéticas.

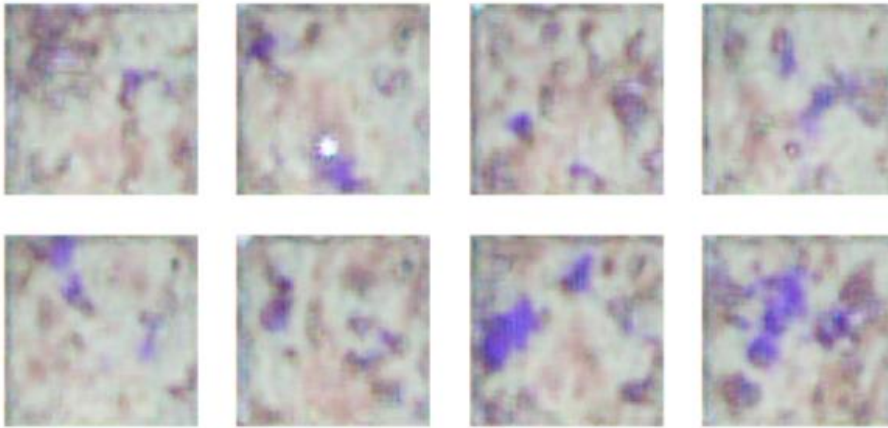
- **Primer ciclo.**

Se empezó con un batch de 64 por 8 épocas, esto para poder observar el tiempo que tomaba cada época con parámetros tan pequeños. El resultado fue únicamente ruido con fondo blanco.



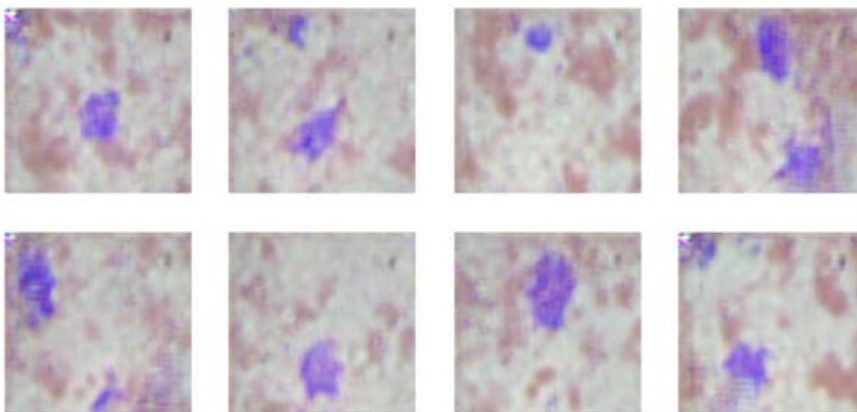
- **Segundo ciclo.**

Batch de 64 por 200 épocas, el resultado muestra aun únicamente ruido con morfologías indistinguibles. Sin embargo, una mejora notoria comienza a verse en los colores.



- **Tercer ciclo.**

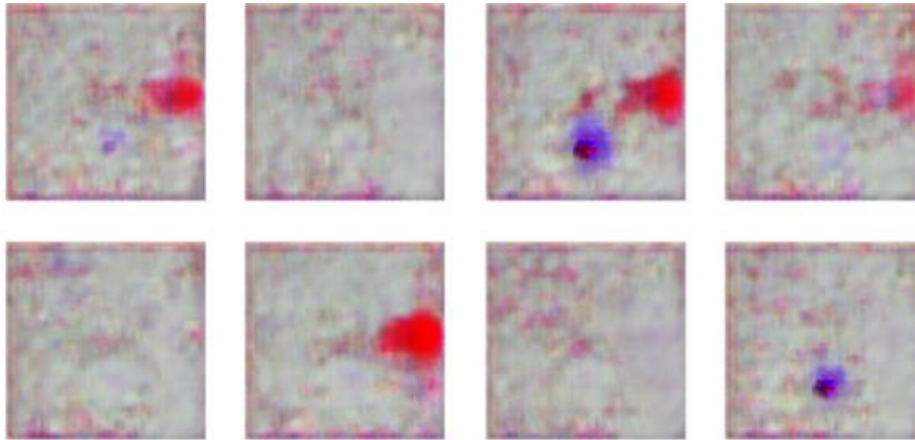
Batch de 64 por 500 épocas, ahora no sólo se han obtenido los colores característicos de las células, sino que, también es posible comenzar a ver cómo se han aglomerado de manera que era claro que se estaban formando las morfologías esperadas.



- **Cuarto ciclo.**

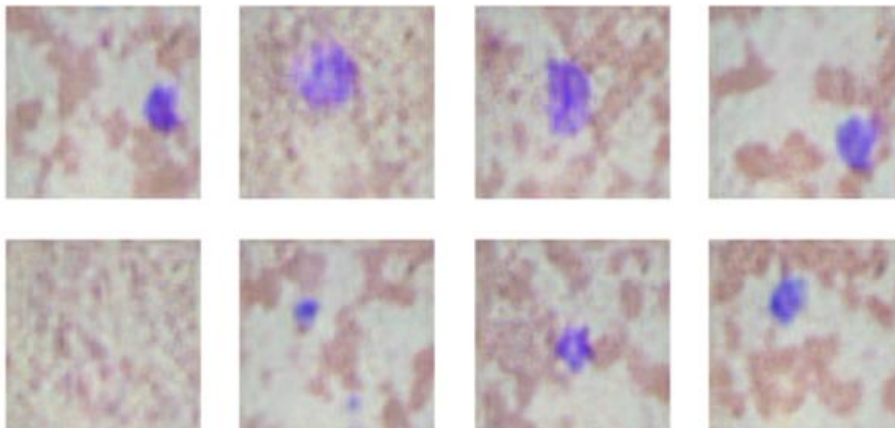
Batch de 128 por 500 épocas, en este punto parece que la red se inestabilizó de nuevo, esto se debe a que al aumentar el número de imágenes en el lote con el cuál se entrenó inicialmente debe aprender a analizar una mayor cantidad de datos. Aun así, los núcleos y colores característicos de las imágenes reales

aparecen, por lo que fue intuitivo seguir con el entrenamiento con el batch aumentado y dejarlo en 128. En los siguientes ciclos únicamente se aumentó el número de épocas.



- **Quinto ciclo.**

Este resultado demostró que el continuar con el ciclo de entrenamiento en el batch de 128 aumentando únicamente a 1200 épocas era lo indicado, es posible ver a una baja resolución morfologías cercanas a agranulocitos, es decir, núcleos morados sin una estructura definida rodeados del frotis de la sangre y el fondo beige de las demás células que acompañan a los leucocitos.



- **Ciclo final.**

En el último ciclo, el entrenamiento se realizó durante 2000 épocas y los resultados fueron mucho mejores a los esperados. En comparación con el artículo en el que se basó este proyecto las personalizaciones mencionadas anteriormente dieron como resultado imágenes con mayor definición tanto en

los núcleos del leucocito, cómo en la parte roja que sería el frotis de la sangre. Así mismo la generación de cuadrículas para mostrar resultados refiere a una mayor cantidad de datos sintetizados en este proyecto, por lo tanto, hay más morfologías con las que comparar imágenes reales y más análisis para propuestas de mejora en la personalización de la red neuronal.

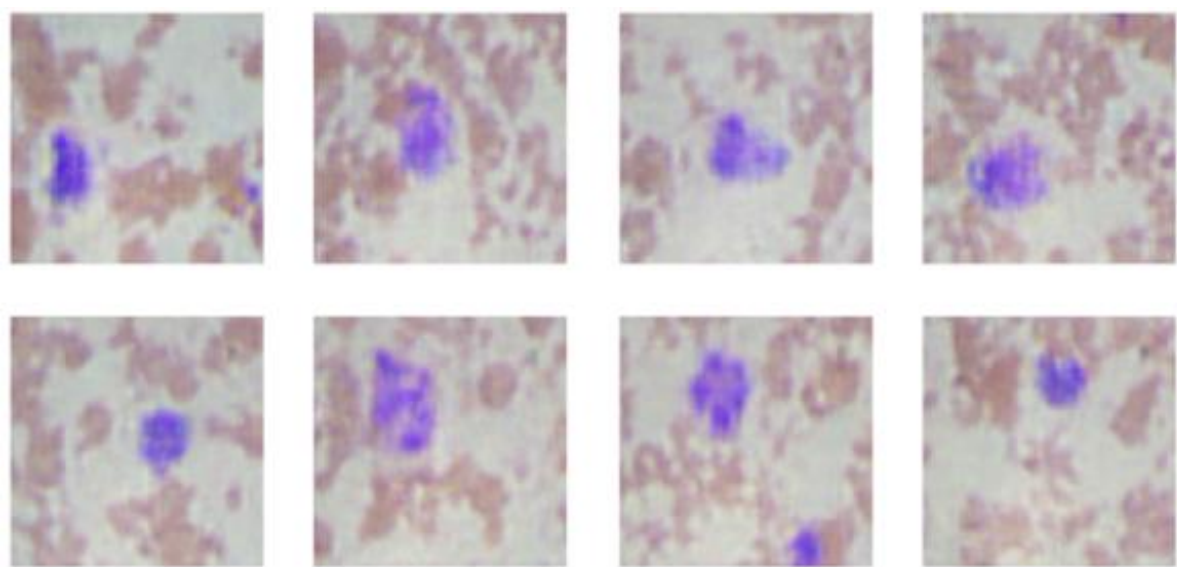


FIGURA CI. RESULTADO DEL ÚLTIMO ENTRENAMIENTO.

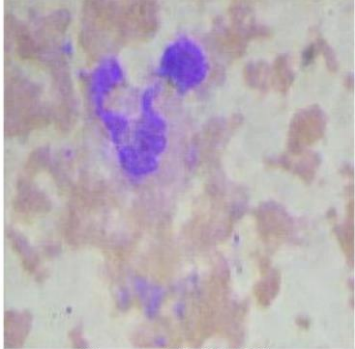
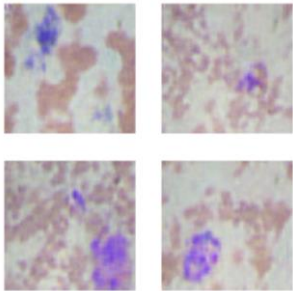
| Real.                                                                               | Referencia.                                                                                                 | Resultado.                                                                            |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|  |  <p>Neutrophils(GAN)</p> |  |

FIGURA CII. COMPARATIVA ENTRE IMÁGENES SINTETIZADAS.

#### IV. Futuros trabajos.

Si bien la arquitectura actual ha demostrado viabilidad en la síntesis morfológica base, es claro que pueden realizarse propuestas para alcanzar un grado clínico de fotorrealismo como puede ser la transición hacia modelos de estado del arte como StyleGAN. La integración a ésta permitiría desacoplar el

espacio latente mediante una red de mapeo y controlar la síntesis a través de múltiples escalas de resolución, posibilitando una modulación altamente precisa tanto de características estructurales macroscópicas (como la compleja geometría de los núcleos multilobulados) y detalles de alta frecuencia, tales como la textura granular del citoplasma o las sutiles variaciones de los reactivos de tinción. Además, también sería factible pensar en una generación de imágenes sintéticas probando arquitecturas condicionales, con las cuales podrían crearse imágenes de subtipos celulares específicos, abarcando las distintas líneas de glóbulos blancos presentes la teoría biológica que se mostró al inicio de este proyecto, subtipos de granulocitos o agranulocitos.

#### ***V. Tabla de actividades.***

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. | Planteamiento del problema (Justificación y objetivo) e idea general.                                                                                                                                                                                                                                                                                                                                                          |
| 2. | Revisión del estado actual del arte. (Lectura de artículos sobre generación de imágenes artificiales de leucocitos con GAN).                                                                                                                                                                                                                                                                                                   |
| 3. | Lectura profunda del reporte “Sangal, R., & Sankaran Krishnan, V. ([2022]). <i>Enhancing white blood cell classification through GAN-based data augmentation</i> [Reporte de proyecto no publicado] “.                                                                                                                                                                                                                         |
| 4. | Búsqueda, obtención, revisión y caracterización del dataset empleado en el reporte.                                                                                                                                                                                                                                                                                                                                            |
| 5. | Revisión detallada del GitHub creado por los autores del reporte.                                                                                                                                                                                                                                                                                                                                                              |
| 6. | Análisis y comparativa del código encontrado en el GitHub referente a la arquitectura Vanilla GAN y lo mencionado en el reporte. Además de búsqueda en la posible mejora de la arquitectura cambiando algunos parámetros.                                                                                                                                                                                                      |
| 7. | Implementación en código del acceso al dataset en Google Colab, el manejo de directorios en Google Drive, guardado de los modelos generados, checkpoints para no perder progreso, el data augmentation, aplicación de la modificación de parámetros, partes del preprocesamiento faltantes, funciones de pérdida, entrenamiento de la Vanilla GAN y detalles sobre la impresión de imágenes para probar el entrenamiento, etc. |
| 8. | Realización y registro de 4 entrenamientos exitosos (uno por cada clase), tomando un batch size de 4 y entrenando con 500 imágenes obtenidas del dataset (ya preprocesado) con una partición de 80% training y 20% dividido un 50%-50% entre validation y test.                                                                                                                                                                |
| 9. | Realización y registro de 6 entrenamientos exitosos, tomando un batch size de 64 y después aumentando a 128. Durante 6 ciclos con aumento de épocas empezando en 8 hasta 2000.                                                                                                                                                                                                                                                 |

|     |                                                                                                                                                    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 10. | Análisis de resultados en base a una comparativa directa con los resultados mostrados en el reporte y las imágenes reales que componen al dataset. |
| 11. | Documentación de códigos empleados.                                                                                                                |
| 12. | Realización de protocolo y presentación del trabajo realizado.                                                                                     |

## VI. Referencias.

Sangal, R., & Sankaran Krishnan, V. (2022)]. *Enhancing white blood cell classification through GAN-based data augmentation* [Reporte de proyecto no publicado].

Aquí están las referencias en texto plano:

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. En *Advances in Neural Information Processing Systems* (pp. 2672–2680).

<https://arxiv.org/abs/1406.2661>

Litjens, G., Kooi, T., Bejnordi, B. E., Setio, A. A. A., Ciompi, F., Ghafoorian, M., van der Laak, J. A. W. M., van Ginneken, B., & Sánchez, C. I. (2017). A survey on deep learning in medical image analysis. *Medical Image Analysis*, 42, 60–88. <https://doi.org/10.1016/j.media.2017.07.005>

Safwan, K. D. B. (s.f.). *Vanilla-GAN* [Código fuente]. GitHub.

<https://github.com/safwankdb/Vanilla-GAN>

Liang, G., Hong, H., Xie, W., & Zheng, L. (2018). Combining convolutional neural network with recursive neural network for blood cell image classification. *IEEE Access*, 6, 36188–36197.

doi:10.1109/ACCESS.2018.2846682

Shen, D., Wu, G., & Suk, H.-I. (2017). Deep learning in medical image analysis. *Annual Review of Biomedical Engineering*, 19, 221–248.

Gopinath, K., Krishnan, M., & Babu, R. V. (2020). A novel GAN-based data augmentation for retinal vessel segmentation. *Journal of Medical Biological Engineering*, 32(23), 17017–17030.